

Non Recursive traversal on Trees

Day 168
25/12/2025

Non Recursive PreOrder Traversal

In the recursive version, a stack is required as we need to remember the current node so that after completing the left subtree we can go to the right subtree.

To simulate the same, first we process the current node and before going to the left subtree, we store current node on stack. after completing the left subtree processing, pop the element and go to its right subtree.

Continue this process until the stack is non empty.

```
public static void preorder (TreeNode root) {
```

```
    if (root == null) return;
```

```
    Stack < TreeNode > st = new Stack < > ();
```

```
    st.push (root);
```

```
    while (!st.isEmpty()) {
```

```
        TreeNode cur = st.pop();
```

```
        System.out.print (cur.data + " ");
```

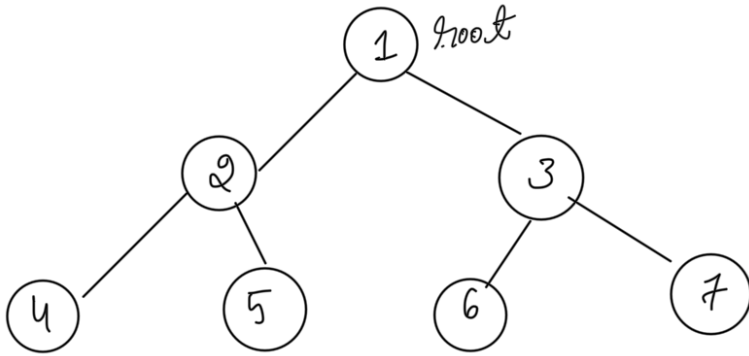
```
        if (cur.right != null) st.push (cur.right);
```

```
        if (cur.left != null) st.push (cur.left);
```

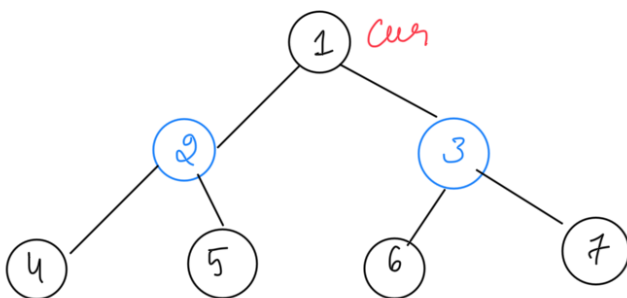
```
    }
```

```
}
```

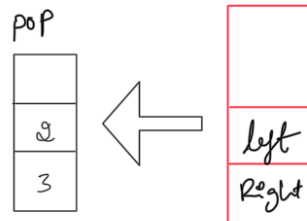
Recursive
Non



st.push (root)

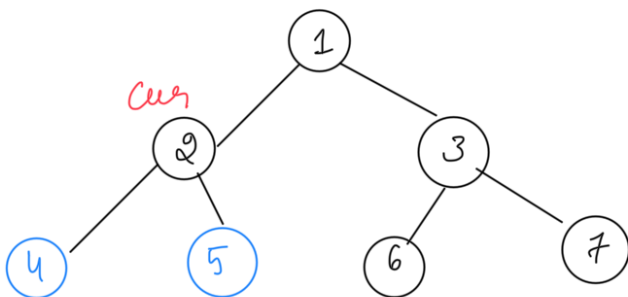


cur = st.pop()
 cur = root

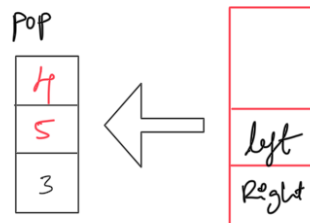


st.push (cur.Right)
 st.push (cur.Left)

preorder 1

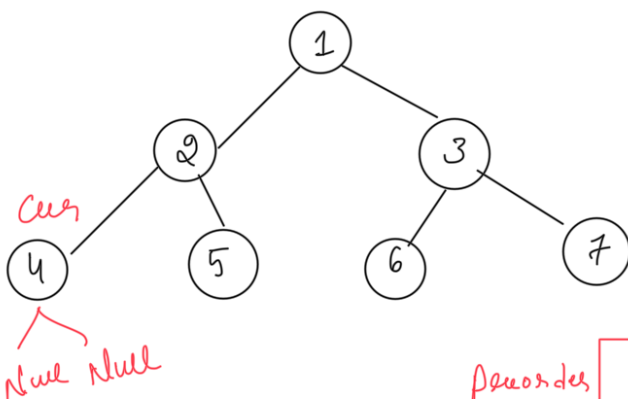


cur = st.pop()
 cur = 2

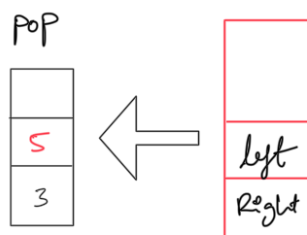


st.push (cur.Right)
 st.push (cur.Left)

preorder 1 2

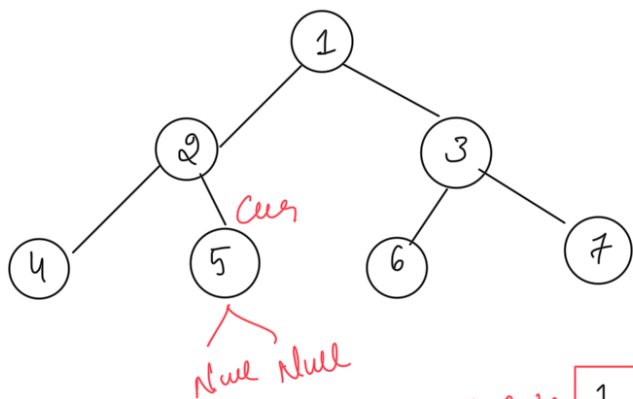


cur = st.pop()
 cur = 4



st.push (cur.Right)
 st.push (cur.Left)

preorder 1 2 4



pop



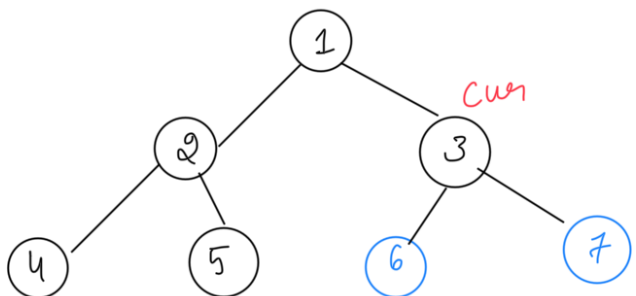
$cur = st.pop()$

$cur = 5$

$st.push(cur.right)$

$st.push(cur.left)$

preOrder 1 2 4 5



pop



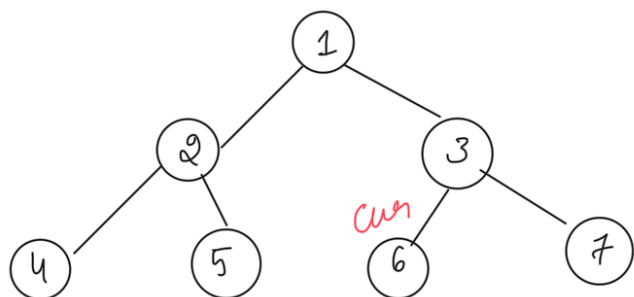
$cur = st.pop()$

$cur = 3$

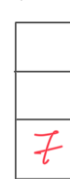
$st.push(cur.right)$

$st.push(cur.left)$

preOrder 1 2 4 5 3



pop



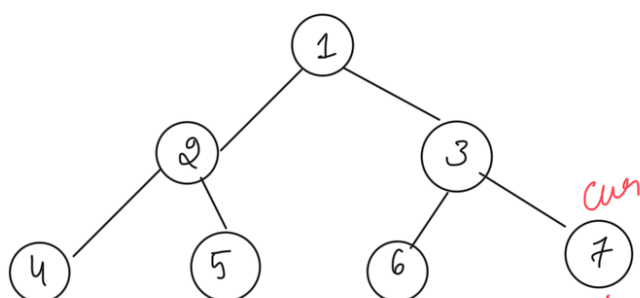
$cur = st.pop()$

$cur = 6$

$st.push(cur.right)$

$st.push(cur.left)$

preOrder 1 2 4 5 3 6



pop



$cur = st.pop()$

$cur = 7$

$st.push(cur.right)$

$st.push(cur.left)$

preOrder 1 2 4 5 3 6 7